access to, so you can't just let them attach to your database. By providing an *application programming interface* (API), your project can be accessed by other systems.

There are two common categories of systems that might use your API: the first is another computer accessing your data, while the second is a user interface. Mobile phone applications on iPhone or Android typically get built with custom software specific to those devices. These applications still need a backend, though, just like your web application does. An API provides a way for mobile apps to talk to your system. Larger projects might have both web and mobile interfaces, and using an API means the user can interact with their data, no matter which frontend they are using.

Another kind of frontend has become popular as well: SPAs. These are still web applications, but instead of the backend producing individual web pages, a single user interface that operates in JavaScript gets downloaded to the browser, and all further UI interaction gets done in place. The *single* in *single-page* means only one web page gets used: a single, bundled JavaScript application. Frameworks like React and Angular are built using this mechanism. Django can still get used as the backend for an SPA, through the use of an API. Of course, the world isn't quite that simple. There are mobile applications built using web technologies and multipage web applications that have interactive JavaScript on a page that also might use an API. Figure 11.1 shows this variety.

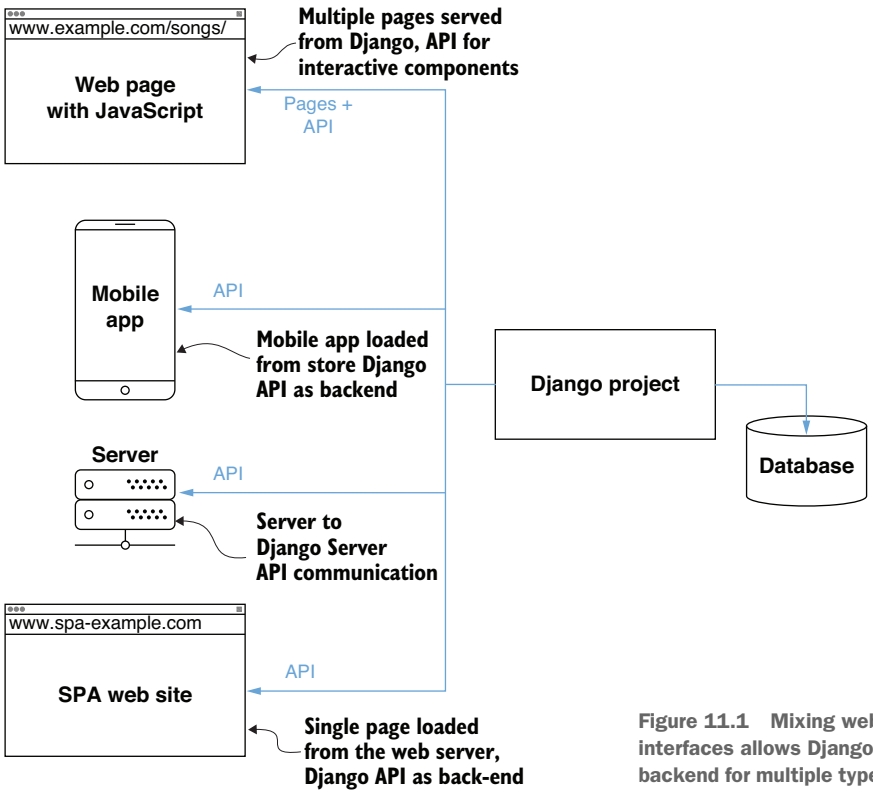


Figure 11.1 Mixing web and API interfaces allows Django to be a backend for multiple types of devices